



Gorgo Go for Java Programmers

May 31, 2026

Contents

15 JSON, HTTP, and the Web	1
Encoding JSON	1
Building an HTTP Server	3
Live Profiling with net/http/pprof	6
Making HTTP Requests	7
Encoding XML	8
Raw Networking with net	9
TLS with crypto/tls	11
Key Points	13
Exercises	13

Chapter 15

JSON, HTTP, and the Web

Go's standard library ships everything you need to build and consume web services — no Spring Boot, no Jackson, no Tomcat required. This chapter covers JSON encoding and decoding, building HTTP servers with the 1.22 mux, writing middleware, live profiling endpoints with `net/http/pprof`, making HTTP requests, XML encoding, raw TCP/UDP networking with the `net` package, and securing connections with `crypto/tls`.

Encoding JSON

Java programmers typically reach for Jackson or Gson to serialize objects to JSON. In Go, encoding/json is in the standard library and needs no configuration beyond struct tags.

The two core functions are:

```
func Marshal(v any) ([]byte, error)           // encode v as JSON bytes
func Unmarshal(data []byte, v any) error      // decode JSON bytes into v (v must be a non-nil pointer)
```

Here is a round trip:

```
package main

import (
    "encoding/json"
    "fmt"
)

type Song struct {
    Title  string `json:"title"`
    Artist string `json:"artist"`
    BPM    int    `json:"bpm"`
}

func main() {
    s := Song{Title: "Todo De Ti", Artist: "Rauw Alejandro", BPM: 96}

    data, err := json.Marshal(s)
    if err != nil {
        panic(err)
    }
    fmt.Println(string(data))
    // {"title":"Todo De Ti","artist":"Rauw Alejandro","bpm":96}
```

```

var s2 Song
if err := json.Unmarshal(data, &s2); err != nil {
    panic(err)
}
fmt.Printf("%+v\n", s2)
// {Title:Todo De Ti Artist:Rauw Alejandro BPM:96}
}

```

Marshal returns a []byte. Convert it to string with string(data) when you need to print it or embed it in a larger string. Unmarshal takes a pointer so it can write into the struct; passing a non-pointer returns an error.



Tip: In Jackson you annotate fields with @JsonProperty("title"). In Go you use a struct tag: `json:"title"`. Both achieve the same mapping — Go's approach requires no annotation processor or reflection framework beyond what encoding/json already does at runtime.

Struct Tags for JSON

A struct tag is a raw string literal attached to a field declaration. encoding/json reads the json key in each tag to control marshalling.

```

type Track struct {
    Title    string `json:"title"           // rename to "title"
    Artist   string `json:"artist"          // rename to "artist"
    BPM      int    `json:"bpm,omitempty"    // omit if BPM == 0
    Internal string `json:"--"              // always skip
}

```

The options after the comma are:

- omitempty — skip the field when its value is the zero value for its type (0, "", false, nil).
- - — never include this field in JSON output, even if it has a value.

```

t1 := Track{Title: "Lokera", Artist: "Rauw Alejandro", BPM: 99, Internal: "secret"}
data, _ := json.Marshal(t1)
fmt.Println(string(data))
// {"title":"Lokera","artist":"Rauw Alejandro","bpm":99}
// Internal is gone; BPM present because it is non-zero

```

```

t2 := Track{Title: "telepatía", Artist: "Kali Uchis"}
data, _ = json.Marshal(t2)
fmt.Println(string(data))
// {"title":"telepatía","artist":"Kali Uchis"}
// BPM omitted because it is zero (omitempty)

```



Trap: omitempty on a struct field type omits it only when the struct pointer is nil, not when the struct is zero-valued. If you want a nested struct to be omissible, make the field a pointer: `json:"meta,omitempty"` on a *Meta field will omit meta when the pointer is nil.

Tag syntax is strict: backtick string, key:"value" pairs separated by spaces, no commas between pairs. go vet ./... catches malformed tags.

Streaming with Encoder and Decoder

Marshal and Unmarshal work on in-memory byte slices. When the JSON is coming from or going to an io.Reader or io.Writer — an HTTP body, a file, a network connection — use json.NewEncoder and

json.NewDecoder instead.

```
func NewEncoder(w io.Writer) *Encoder // write JSON to w
func NewDecoder(r io.Reader) *Decoder // read JSON from r
```

Key methods on those types:

```
func (enc *Encoder) Encode(v any) error // marshal v and write to w, followed by a newline
func (dec *Decoder) Decode(v any) error // read one JSON value from r and unmarshal into v
func (dec *Decoder) More() bool        // true if another value remains in the stream
```

Encoding directly to an http.ResponseWriter:

```
func songHandler(w http.ResponseWriter, r *http.Request) {
    s := Song{Title: "I Wish You Roses", Artist: "Kali Uchis", BPM: 90}
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(s)
}
```

Decoding a stream of newline-delimited JSON records:

```
dec := json.NewDecoder(r.Body)
for dec.More() {
    var s Song
    if err := dec.Decode(&s); err != nil {
        break
    }
    fmt.Println(s.Title)
}
```



Tip: Prefer streaming Encoder/Decoder over Marshal/Unmarshal when working with io.Reader/io.Writer. Streaming avoids allocating the entire JSON payload as a byte slice, which matters for large payloads.

Building an HTTP Server

Chapter 8 introduced http.HandlerFunc as an example of the strategy pattern. Here is the full picture.

The foundation of Go's HTTP server is the http.Handler interface:

```
type Handler interface {
    ServeHTTP(ResponseWriter, *Request) // handles one HTTP request
}
```

http.HandlerFunc is a function type that implements Handler by calling itself (Chapter 8). The convenience wrapper http.HandleFunc registers a plain function with the default mux:

```
func HandleFunc(pattern string, handler func(ResponseWriter, *Request)) // register handler for pattern
func ListenAndServe(addr string, handler Handler) error                 // start the server; handler=nil
```

A minimal server:

```
package main

import (
    "fmt"
    "net/http"
)
```

```
func helloHandler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "Hola!\n")
}

func main() {
    http.HandleFunc("/hello", helloHandler)
    http.ListenAndServe(":8080", nil)
}
```



Trap: `http.ListenAndServe` blocks the calling goroutine forever (or until the server fails). If you want to do other work in `main`, launch the server in a goroutine: `go http.ListenAndServe(":8080", nil)`. Check the returned error in production code — a common pattern is `log.Fatal(http.ListenAndServe(":8080", nil))`.

Go 1.22 ServeMux — Method Routing and Wildcards

Before Go 1.22, `http.ServeMux` matched only on path prefixes and required external routers (Chi, Gorilla Mux, etc.) for method or wildcard routing. Go 1.22 upgraded the built-in mux significantly.

Method routing — prefix the pattern with an HTTP method and a space:

```
mux := http.NewServeMux()
mux.HandleFunc("GET /songs/", listSongs)    // only GET requests
mux.HandleFunc("POST /songs/", createSong)  // only POST requests
```

Path wildcards — `{name}` captures a path segment, `{name...}` captures the rest:

```
mux.HandleFunc("GET /songs/{id}/", getSong) // /songs/42/ --- id = "42"
```

Retrieve the captured value with `r.PathValue`:

```
func getSong(w http.ResponseWriter, r *http.Request) {
    id := r.PathValue("id")           // "42"
    fmt.Fprintf(w, "fetching song %s\n", id)
}
```

A complete example wiring up a small songs API:

```
package main

import (
    "encoding/json"
    "net/http"
)

type Song struct {
    ID      string `json:"id"`
    Title   string `json:"title"`
    Artist  string `json:"artist"`
}

var catalog = map[string]Song{
    "1": {ID: "1", Title: "Todo De Ti", Artist: "Rauw Alejandro"},
    "2": {ID: "2", Title: "I Wish You Roses", Artist: "Kali Uchis"},
}

func getSong(w http.ResponseWriter, r *http.Request) {
```

```

    id := r.PathValue("id")
    song, ok := catalog[id]
    if !ok {
        http.Error(w, "not found", http.StatusNotFound)
        return
    }
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(song)
}

func listSongs(w http.ResponseWriter, r *http.Request) {
    songs := make([]Song, 0, len(catalog))
    for _, s := range catalog {
        songs = append(songs, s)
    }
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(songs)
}

func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("GET /songs/", listSongs)
    mux.HandleFunc("GET /songs/{id}/", getSong)
    http.ListenAndServe(":8080", mux)
}

```

Notice that `getSong` calls `http.Error` and immediately returns when the song is not found, so the success path only runs when the lookup succeeds. [*error-first-return-early*]



Wut: A trailing slash in the pattern makes it a subtree match: `GET /songs/` matches `/songs/`, `/songs/anything`, and `/songs/42/`. Without the trailing slash, `GET /songs` matches only the exact path `/songs`. This is the same behavior as pre-1.22 `ServeMux`.

Compared to Java: this is the rough equivalent of Spring MVC's `@GetMapping("/songs/{id}")` or JAX-RS's `@GET @Path("/songs/{id}")`, but built into the standard library with no framework dependency.

Middleware Chaining

Middleware in Go is a function that takes an `http.Handler` and returns an `http.Handler`. It wraps the inner handler with cross-cutting logic — logging, authentication, metrics, CORS — without modifying the handler itself.

The pattern:

```

func logging(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        log.Printf("%s %s", r.Method, r.URL.Path) // before
        next.ServeHTTP(w, r)                     // call the inner handler
    })
}

```

Stack middleware by composing the wrappers:

```

func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("GET /songs/{id}/", getSong)
}

```

```

    handler := logging(mux) // outermost middleware runs first
    http.ListenAndServe(":8080", handler)
}

```

Add more layers by nesting calls:

```

handler := logging(auth(mux)) // logging wraps auth wraps mux

```



Tip: The middleware pattern is the Go equivalent of Java's servlet filters or Spring's `HandlerInterceptor`. Because `http.Handler` is a simple interface with one method, any function that wraps it composes cleanly — no framework annotation or XML configuration required.

You can also write a chain helper to apply a list of middleware in order:

```

func chain(h http.Handler, middleware ...func(http.Handler) http.Handler) http.Handler {
    for i := len(middleware) - 1; i >= 0; i-- {
        h = middleware[i](h) // apply in reverse so the first listed runs first
    }
    return h
}

```

```

handler := chain(mux, logging, auth, metrics)

```

Live Profiling with `net/http/pprof`

Go's runtime can expose profiling data — CPU usage, heap allocations, goroutine stacks, mutex contention — as HTTP endpoints. All it takes is a blank import.

```

import _ "net/http/pprof" // register pprof handlers with DefaultServeMux

```

The side-effect import registers several routes under `/debug/pprof/` on `http.DefaultServeMux`. If your server already calls `http.ListenAndServe(addr, nil)` (which uses `DefaultServeMux`), those routes are live the moment the process starts — no code changes beyond the import.

If you are using a custom `*http.ServeMux`, register the handlers explicitly:

```

import (
    "net/http"
    "net/http/pprof"
)

mux := http.NewServeMux()
mux.HandleFunc("GET /songs/{id}/", getSong)

// register pprof endpoints on the same mux
mux.HandleFunc("GET /debug/pprof/",      pprof.Index)
mux.HandleFunc("GET /debug/pprof/cmdline", pprof.Cmdline)
mux.HandleFunc("GET /debug/pprof/profile", pprof.Profile)
mux.HandleFunc("GET /debug/pprof/symbol",  pprof.Symbol)
mux.HandleFunc("GET /debug/pprof/trace",   pprof.Trace)

http.ListenAndServe(":8080", mux)

```

The key endpoints under `/debug/pprof/`:

Endpoint	What it shows
/	index of available profiles
heap	live heap allocations
goroutine	stack trace of every live goroutine
allocs	all past allocations, including freed ones
mutex	stack traces of goroutines holding a mutex
block	stack traces of goroutines blocked on channels
profile?seconds=N	CPU profile for N seconds (default 30)

Once the server is running, collect and view a profile with `go tool pprof`:

```
# collect a 30-second CPU profile and open the interactive browser UI
go tool pprof -http=:6060 http://localhost:8080/debug/pprof/profile?seconds=30
```

```
# inspect the live heap
go tool pprof -http=:6060 http://localhost:8080/debug/pprof/heap
```

`go tool pprof -http` opens a flame graph and call-graph view in your browser. The flame graph makes it easy to see which functions consume the most CPU or allocate the most memory.



Trap: Never expose `/debug/pprof/` on a public-facing port. A CPU profile request ties up a real OS thread for its full duration, making it trivial for an attacker to saturate the server. The standard pattern is to run `pprof` on a separate, internal-only port:

```
// internal admin server on a non-public port
go func() {
    log.Println(http.ListenAndServe("localhost:6060", nil)) // DefaultServeMux has pprof routes
}()
// public server with a custom mux
http.ListenAndServe(":8080", mux)
In a container environment, expose port 6060 only within the cluster network.
```



Tip: In Java, runtime profiling typically requires attaching an external tool (VisualVM, JProfiler, YourKit) via the JVM attach mechanism or JMX. `net/http/pprof` is the Go equivalent built into the process — no agent, no attach, no separate daemon. The profile data is available over plain HTTP, which means you can script it with `curl` or integrate it into any monitoring pipeline.

Making HTTP Requests

The `net/http` package is also the HTTP client. The simplest way to make a GET request is `http.Get`:

```
func Get(url string) (resp *Response, err error) // make a GET request; caller must close resp.Body
resp, err := http.Get("https://api.example.com/songs/1")
if err != nil {
    log.Fatal(err)
}
defer resp.Body.Close() // MUST close the body

body, err := io.ReadAll(resp.Body)
if err != nil {
    log.Fatal(err)
}
fmt.Println(string(body))
```



Trap: You **must** close `resp.Body`, even if you do not read it. Failing to close the body leaks the underlying TCP connection and eventually exhausts the connection pool. `defer resp.Body.Close()` immediately after checking `err` is the standard idiom.

For anything beyond a simple GET — custom headers, timeouts, POST bodies — use `http.Client` and `http.NewRequest`:

```
// http.Client --- reusable, configurable; zero value uses sensible defaults
type Client struct {
    Transport RoundTripper // how to make a single HTTP transaction
    CheckRedirect func(req *Request, via []*Request) error // redirect policy
    Jar CookieJar // cookie storage
    Timeout time.Duration // zero means no timeout
}

func NewRequest(method, url string, body io.Reader) (*Request, error) // build a request
func (c *Client) Do(req *Request) (*Response, error) // execute the request
```

A POST with a JSON body and a timeout:

```
client := &http.Client{Timeout: 10 * time.Second}

song := Song{Title: "Lokera", Artist: "Rauw Alejandro", BPM: 99}
buf := new(bytes.Buffer)
json.NewEncoder(buf).Encode(song)

req, err := http.NewRequest("POST", "https://api.example.com/songs/", buf)
if err != nil {
    log.Fatal(err)
}
req.Header.Set("Content-Type", "application/json")

resp, err := client.Do(req)
if err != nil {
    log.Fatal(err)
}
defer resp.Body.Close()

fmt.Println(resp.StatusCode) // e.g. 201
```



Tip: Create one `http.Client` and reuse it across requests. `http.Client` manages a connection pool (Transport) internally. Creating a new client per request bypasses pooling and creates a new pool each time, which is wasteful and slower. The package-level `http.Get` and `http.Post` helpers use a shared default client (`http.DefaultClient`).

In Java you would use `HttpClient` (Java 11+), `RestTemplate`, or `OkHttp`. Go's `http.Client` fills the same role.

Encoding XML

`encoding/xml` works exactly like `encoding/json` — same `Marshal/Unmarshal` functions, same streaming `Encoder/Decoder`, same struct tag mechanism. The tag key is `xml` instead of `json`.

```
func Marshal(v any) ([]byte, error) // encode v as XML
func Unmarshal(data []byte, v any) error // decode XML into v
```

```

func NewEncoder(w io.Writer) *Encoder // streaming XML encoder
func NewDecoder(r io.Reader) *Decoder // streaming XML decoder

import "encoding/xml"

type Song struct {
    XMLName xml.Name `xml:"song"` // set the element name
    Title   string   `xml:"title"`
    Artist  string   `xml:"artist"`
    BPM     int      `xml:"bpm,omitempty"`
}

s := Song{Title: "telepatía", Artist: "Kali Uchis", BPM: 90}
data, _ := xml.Marshal(s)
fmt.Println(string(data))
// <song><title>telepatía</title><artist>Kali Uchis</artist><bpm>90</bpm></song>

```

The `xml.Name` field with tag ``xml:"song"`` sets the root element name. Without it, `encoding/xml` uses the struct type name, which is often not what you want.

XML tag options beyond the field name:

- `xml:"name,attr"` — encode the field as an XML attribute rather than a child element.
- `xml:",chardata"` — encode the field as the character data content of the parent element.
- `xml:",omitempty"` — same as JSON: omit when zero value.
- `xml:"-"` — always skip.



Tip: If your service needs both JSON and XML representations of the same struct, define both tag keys on each field:

```

type Song struct {
    Title string `json:"title" xml:"title"`
    Artist string `json:"artist" xml:"artist"`
}

```

Both `encoding/json` and `encoding/xml` read only their own tag key and ignore the other.

Raw Networking with net

HTTP sits on top of TCP. The `net` package gives you direct access to TCP and UDP connections — useful for non-HTTP protocols, custom servers, or low-level networking tools.

The two fundamental functions are:

```

func Dial(network, address string) (Conn, error) // connect to a server
func Listen(network, address string) (Listener, error) // listen for incoming connections

```

`network` is "tcp", "tcp4", "tcp6", "udp", "udp4", or "udp6". `address` is "host:port" for TCP/UDP, e.g. "localhost:8080" or ":8080" (all interfaces).

`net.Conn` implements both `io.Reader` and `io.Writer`, so you can use any `io` utility on it:

```

// key methods on net.Conn
func (c Conn) Read(b []byte) (n int, err error) // receive bytes
func (c Conn) Write(b []byte) (n int, err error) // send bytes
func (c Conn) Close() error // close the connection
func (c Conn) SetDeadline(t time.Time) error // set read+write deadline
func (c Conn) SetReadDeadline(t time.Time) error // set read deadline only
func (c Conn) SetWriteDeadline(t time.Time) error // set write deadline only

```

A simple TCP echo client:

```
package main

import (
    "fmt"
    "net"
)

func main() {
    conn, err := net.Dial("tcp", "localhost:9000")
    if err != nil {
        panic(err)
    }
    defer conn.Close()

    fmt.Fprintf(conn, "hola\n")    // send

    buf := make([]byte, 1024)
    n, _ := conn.Read(buf)        // receive
    fmt.Printf("echo: %s", buf[:n])
}
```

A minimal TCP echo server:

```
package main

import (
    "io"
    "net"
)

func main() {
    ln, err := net.Listen("tcp", ":9000")
    if err != nil {
        panic(err)
    }
    defer ln.Close()

    for {
        conn, err := ln.Accept()    // block until a client connects
        if err != nil {
            break
        }
        go io.Copy(conn, conn)      // echo: copy reads back to writes
    }
}
```

`ln.Accept()` blocks until a new connection arrives and returns a `net.Conn` for that connection. Each connection is handled in its own goroutine so the server can accept the next connection immediately.



Wut: `io.Copy(conn, conn)` works because `net.Conn` implements both `io.Reader` and `io.Writer`. `io.Copy(dst, src)` reads from `src` and writes to `dst` until EOF or an error. Here `conn` is both — every byte received is immediately written back.

For UDP, the workflow is the same but with `net.Dial("udp", ...)` and `net.Listen("udp", ...)`. UDP

has no connection setup, so `net.ListenPacket` and `net.PacketConn` are sometimes used instead for servers that need to track multiple clients by address.



Tip: In Java, raw TCP/UDP programming uses `java.net.Socket`, `ServerSocket`, `DatagramSocket`, and `DatagramPacket`. Go's `net.Conn` is a simpler unified abstraction: it is an `io.Reader` and `io.Writer`, so you can layer a `bufio.Scanner` or a `json.Decoder` directly on top of it without any adapter code.

TLS with crypto/tls

The `crypto/tls` package provides TLS 1.2 and 1.3 support for both servers and clients. For HTTP, the standard library handles TLS transparently — you mostly just point it at your certificate files. For raw TCP, `crypto/tls` wraps a `net.Conn` into an encrypted connection that still satisfies `io.Reader` and `io.Writer`.

HTTPS Server

The simplest way to add TLS to an HTTP server is `http.ListenAndServeTLS`:

```
func ListenAndServeTLS(addr, certFile, keyFile string, handler Handler) error
```

```
mux := http.NewServeMux()
mux.HandleFunc("GET /songs/", listSongs)
http.ListenAndServeTLS(":443", "cert.pem", "key.pem", mux)
```

For more control, build a `tls.Config` and pass it to `http.Server`:

```
cert, err := tls.LoadX509KeyPair("cert.pem", "key.pem") // load certificate and private key
if err != nil {
    log.Fatal(err)
}
srv := &http.Server{
    Addr:    ":443",
    Handler: mux,
    TLSConfig: &tls.Config{
        Certificates: []tls.Certificate{cert}, // one or more certificates
        MinVersion:   tls.VersionTLS12,       // reject TLS 1.0 and 1.1
    },
}
srv.ListenAndServeTLS("", "") // cert and key already in TLSConfig; pass empty strings
```

TLS Client Configuration

The default `http.Client` already validates server certificates against the system trust store — you get HTTPS for free just by using an `https://` URL. To customize TLS behavior, set `TLSClientConfig` on `http.Transport`:

```
client := &http.Client{
    Transport: &http.Transport{
        TLSClientConfig: &tls.Config{
            MinVersion: tls.VersionTLS12, // require TLS 1.2+
        },
    },
}
```

To trust a custom certificate authority (common in internal services):

```
pool := x509.NewCertPool()
caCert, _ := os.ReadFile("internal-ca.pem")
pool.AppendCertsFromPEM(caCert)

client := &http.Client{
    Transport: &http.Transport{
        TLSClientConfig: &tls.Config{
            RootCAs: pool, // trust this CA in addition to the system store
        },
    },
}
```



Trap: `tls.Config{InsecureSkipVerify: true}` disables all certificate validation. It is occasionally used in local development against self-signed certs, but it silently makes man-in-the-middle attacks undetectable. Never use it in production code, and never commit it to a shared repository.

Raw TLS Connections

`crypto/tls` mirrors the `net` package: `tls.Dial` and `tls.Listen` replace `net.Dial` and `net.Listen` and return a `*tls.Conn`, which implements `net.Conn` — and therefore `io.Reader` and `io.Writer`.

```
func Dial(network, addr string, config *tls.Config) (*tls.Conn, error) // TLS client connection
func Listen(network, laddr string, config *tls.Config) (net.Listener, error) // TLS server listener
```

A TLS client connecting to a server:

```
conn, err := tls.Dial("tcp", "api.example.com:443", &tls.Config{
    MinVersion: tls.VersionTLS12,
})
if err != nil {
    log.Fatal(err)
}
defer conn.Close()

fmt.Fprintf(conn, "GET / HTTP/1.0\r\nHost: api.example.com\r\n\r\n")
io.Copy(os.Stdout, conn)
```

`tls.NewListener` wraps an existing `net.Listener` to add TLS to any protocol:

```
func NewListener(inner net.Listener, config *tls.Config) net.Listener

ln, _ := net.Listen("tcp", ":9443")
tlsLn := tls.NewListener(ln, &tls.Config{Certificates: []tls.Certificate{cert}})
for {
    conn, _ := tlsLn.Accept() // conn is a *tls.Conn wrapping a net.Conn
    go handleConn(conn)
}
```



Tip: In Java, TLS requires `SSLContext`, `KeyManagerFactory`, `TrustManagerFactory`, and an `SSLSocketFactory` or `SSLServerSocketFactory` — a ceremony that takes dozens of lines even for the common case. Go's `crypto/tls` collapses this to a `tls.Config` struct and a single function call. For HTTPS, `http.ListenAndServeTLS` reduces it further to two file paths.

Key Points

- `json.Marshal` encodes a Go value to `[]byte`; `json.Unmarshal` decodes `[]byte` into a pointer.
- Struct tags control JSON field names, `omitempty` (skip zero values), and `-` (always skip).
- Use streaming `json.NewEncoder/json.NewDecoder` when reading from or writing to an `io.Reader/io.Writer` to avoid allocating the entire payload.
- `http.Handler` is the core interface; `http.HandlerFunc` adapts a plain function to satisfy it (Chapter 8 pattern).
- Go 1.22 `ServeMux` supports method routing (`GET /path/`) and path wildcards (`/songs/{id}/`) in the standard library — no third-party router needed for most services.
- Middleware is a function `func(http.Handler) http.Handler`; compose layers by wrapping.
- Always close `resp.Body` after an HTTP client response — `defer resp.Body.Close()` immediately after the error check.
- Reuse `http.Client` across requests; it manages a connection pool internally.
- `import _ "net/http/pprof"` registers live profiling endpoints on `DefaultServeMux`; use `go tool pprof -http` to analyze; never expose these on a public port.
- `encoding/xml` mirrors `encoding/json` exactly — same `Marshal/Unmarshal` API, same tag mechanism, different key (xml instead of json).
- `net.Dial` and `net.Listen` give raw TCP/UDP access; `net.Conn` implements `io.Reader` and `io.Writer`, so all `io` utilities compose with it.
- `http.ListenAndServeTLS` adds TLS to an HTTP server with two file paths; `tls.Config` controls certificate loading, minimum version, and custom CA trust.
- `tls.Dial` and `tls.Listen` mirror `net.Dial/net.Listen` for raw TLS; `tls.Conn` satisfies `net.Conn` so existing `io` code needs no changes.
- Never use `InsecureSkipVerify: true` in production — it disables all certificate validation.

Exercises

1. **Think about it:** In Java with Spring MVC or JAX-RS, you annotate a class method with `@GetMapping("/songs/{id}")` or `@GET @Path("/songs/{id}")` and the framework discovers handlers via reflection and classpath scanning. In Go, you call `mux.HandleFunc("GET /songs/{id}/", getSong)` explicitly in `main`. What are the tradeoffs of each approach? Consider startup time, debuggability, IDE navigation, and what happens when two handlers are registered for the same pattern.
2. **What does this print?**

```
package main

import (
    "encoding/json"
    "fmt"
)

type Artist struct {
    Name      string `json:"name"`
    Country   string `json:"country,omitempty"`
    Secret    string `json:"-"`
}

func main() {
    a := Artist{Name: "Kali Uchis", Country: "", Secret: "Colombia"}
    data, _ := json.Marshal(a)
    fmt.Println(string(data))
}
```

```

var b Artist
json.Unmarshal([]byte(`{"name":"Rauw Alejandro","secret":"Puerto Rico"}`), &b)
fmt.Printf("Name: %s, Secret: %q\n", b.Name, b.Secret)
}

```

3. **Calculation:** Consider the following ServeMux registration and the three incoming requests below. For each request, state which handler function is called, or 404 if none matches.

```

mux := http.NewServeMux()
mux.HandleFunc("GET /tracks/", listTracks)
mux.HandleFunc("GET /tracks/{id}/", getTrack)
mux.HandleFunc("POST /tracks/", createTrack)

```

- GET /tracks/
- GET /tracks/42/
- DELETE /tracks/7/

4. **Where is the bug?**

```

package main

import (
    "fmt"
    "io"
    "net/http"
)

func fetchLyrics(url string) (string, error) {
    resp, err := http.Get(url)
    if err != nil {
        return "", err
    }
    body, err := io.ReadAll(resp.Body)
    if err != nil {
        return "", err
    }
    return string(body), nil
}

func main() {
    lyrics, err := fetchLyrics("https://api.example.com/lyrics/todo-de-ti")
    if err != nil {
        fmt.Println("error:", err)
        return
    }
    fmt.Println(lyrics)
}

```

5. **Write a program:** Build a small in-memory HTTP server that manages a list of songs. Define a Song struct with fields ID int, Title string, and Artist string, all with appropriate json tags. Store songs in a package-level map[int]Song. Register two routes using a http.NewServeMux():

- GET /songs/ returns all songs as a JSON array.
- GET /songs/{id}/ returns the single song with that ID, or HTTP 404 if not found. Pre-populate the map with two songs (use Rauw Alejandro or Kali Uchis tracks). Start the server on :8080.